VANE: RELIABLE TEST-TIME TRAINING FOR VISION-LANGUAGE-ACTION MODELS VIA FUTURE VISUAL REPRESENTATION PREDICTION

Hongjin Ji[1,5*] **Guoyang Xia**[2,5,*] **Luoyang Sun**[3,4,5,*] **Fangxiang Feng**[2] **Lei Ren**[5,†,‡]

[1]The Chinese University of Hong Kong, Shenzhen [2]Beijing University of Posts and Telecommunications
[3]Institute of Automation, Chinese Academy of Sciences
[4]University of Chinese Academy of Sciences
[5]Li Auto Inc.

ABSTRACT

Test-time training (TTT) offers a lightweight way to adapt vision–language–action (VLA) policies from unlabeled deployment streams, but it remains difficult to use reliably in closed-loop manipulation. A shared adaptation space can mix incompatible task corrections, while an online update can alter subsequent actions before its consequences are known. We introduce a reliable TTT framework for VLA policies (*VANE*). VANE conditions prompt adaptation on the current vision–language context and learns from the future visual consequences of executed actions. Candidate updates are isolated from the live policy, evaluated on subsequent observations, and committed only when supported by future evidence, making adaptation selective and reversible. On SimplerEnv WidowX, VANE improves average success by 3.2 percentage points over the corresponding TTT baseline. Results on Google Robot further show that deployment-time gains remain task- and embodiment-dependent. Together, these results demonstrate a constrained, evidence-based approach to adapting VLA policies during interaction.

1 INTRODUCTION

An expanding family of vision–language–action (VLA) models provides strong policy backbones for robot manipulation [1–4]. Nevertheless, adapting these backbones to new tasks or visual environments remains expensive even when the robot embodiment is fixed. Conventional adaptation commonly requires additional task-specific demonstrations with action labels, followed by repeated fine-tuning and hyperparameter selection. As the number of downstream tasks grows, this process repeatedly consumes data and engineering effort and often produces a collection of separately tuned policies rather than one maintainable deployed system. In contrast, robot deployment naturally produces streams of unlabeled visual observations. A practical adaptation mechanism should exploit this readily available supervision, preserve a shared VLA backbone, represent task-dependent corrections, and prevent harmful updates from directly entering the closed-loop controller.

Test-time training (TTT) offers a promising alternative by optimizing a self-supervised objective on unlabeled test observations while updating only a small parameter subset [5]. TTT-VLA [6] further demonstrates that a latent prompt can connect a self-supervised proxy task to a frozen VLA policy, avoiding full-model fine-tuning during deployment. This establishes the feasibility of lightweight VLA TTT, but does not yet answer whether the resulting updates remain dependable across heterogeneous tasks and closed-loop interaction. In our controlled study on QwenPi [4], latent-prompt TTT produces inconsistent outcomes across tasks and visual conditions: an update that benefits one setting may degrade another, while an unvalidated online update may reduce the performance of the deployed policy. In this work, we call TTT *reliable* when candidate updates are introduced selectively and regressions caused by task interference or erroneous online optimization are suppressed. We identify three design challenges to achieving this behavior.

*Equal contribution. †Project leader. ‡Corresponding author.
Email: hongjinji@link.cuhk.edu.cn

First, prompt adaptation is strongly task-dependent. At a fixed checkpoint, the non-adapted policy obtains a success rate of 64.1%. Optimizing one prompt jointly over all four tasks decreases the result to 63.3%, whereas independently adapted task-specific prompts reach 66.1%. The gap indicates that a single shared prompt entangles heterogeneous, and sometimes conflicting, correction directions. At the same time, fully independent prompts discard potentially reusable structure across related tasks. We therefore seek an adaptation space that preserves sharing while allowing the current vision–language context to compose different corrections. This motivates a context-routed *Mixture of Latent Prompts* (MoLP), which constructs the effective prompt from a shared latent-prompt bank without requiring an externally supplied task identifier. We analyze the underlying task-sharing interference in Sec. 4.1.1 and evaluate the resulting prompt structure in Sec. 4.2.3.

Second, the usefulness of prompt-only TTT depends critically on its proxy objective. A suitable deployment proxy should satisfy two requirements. First, its target must be obtainable from ordinary robot interaction during both training and deployment, without expert actions, rewards, success labels, or privileged evaluation signals. Second, optimizing it should provide a policy-relevant adaptation signal: the target should preserve semantic structure that can generalize across tasks and visual conditions while exposing interaction-dependent change rather than merely reconstructing what is already visible. State grounding satisfies the availability requirement because the current robot state is directly observed, but its target is low-dimensional, embodiment-specific, and synchronized with the current image. It can therefore be optimized without explicitly explaining how the visual scene evolves after interaction. Delayed RGB observations are equally label-free, and a frozen video representation encoder converts them into semantically structured future visual targets without requiring pixel-level generation. Predicting such a target from the current observation, adaptable prompt, and action context yields a temporally predictive and action-conditioned proxy. We call this supervision branch the *World-Predictive Interface* (WPI).

We instantiate this proxy design on *QwenPi*, a Qwen-based flow-matching VLA constructed within the StarVLA-α framework, rather than on the native $\pi_{0.5}$ policy used by TTT-VLA. We retain the QwenPi action-policy backbone and replace its state-grounding proxy branch with WPI; the resulting variant is called *QwenWPI*. Here, *world-predictive* denotes a world-model-like supervision interface that predicts future visual representations. QwenWPI is not a standalone world model and does not replace the action policy. Matched comparisons between state grounding and WPI are reported in Sec. 4.2.3.

Third, replacing a same-time state proxy with future-representation prediction changes the online optimization problem itself. With state-grounded QwenPi, a low-dimensional proxy target is available at the current step and a prompt update can be formed immediately. With QwenWPI, a valid objective becomes available only after the delayed observation o_{t+k} arrives, and each pair requires encoding and comparing token-level visual representations. More importantly, immediately deploying a candidate prompt changes the actions that generate its subsequent supervision: the update can alter the very future observations on which it would later be judged. Reliable predictive TTT must therefore decide not only when an update is informative, but also how to evaluate it without allowing the candidate to control the trajectory used for validation.

We address this difficulty with *Attention-Gated and Validation-Driven Test-Time Training* (AGV-TTT). Cross-layer attention redistribution provides a policy-internal signal for proposing a sparse update near interaction transitions. The update is applied to a shadow copy of the latent-prompt bank, while the live prompt continues to control the robot. As subsequent observations arrive, the old and candidate prompts are evaluated on matched future-prediction pairs. The candidate prompt and its optimizer state are committed atomically only when the focused objective improves without global regression; otherwise, the candidate is discarded. Attention therefore determines *when to propose* adaptation, while delayed observations determine *whether the proposal should be deployed*. The attention pattern motivating the proposal cue is examined in Sec. 4.1.2, and the contribution of future validation is isolated in Sec. 4.2.4.

These considerations lead to *VANE*, which combines a context-routed latent-prompt adaptation space, a future-predictive proxy, and a selective proposal–validation–commit protocol.The name VANE evokes a weather vane, which responds to changes in its surroundings while remaining anchored to a stable structure, reflecting our goal of selectively adapting a shared policy to changing deployment conditions. MoLP represents task-dependent corrections, QwenWPI supplies observable and policy-relevant self-supervision,and AGV-TTT handles the delayed and closed-loop nature of predictive TTT. At deployment, all policy, routing, and predictive modules remain frozen; only the latent-prompt bank is eligible for validated updates.

Our contributions are summarized as follows:

- We systematically diagnose latent-prompt TTT on QwenPi and show that heterogeneous manipulation tasks require context-dependent corrections. We propose a context-routed Mixture of Latent Prompts (MoLP) that balances cross-task sharing and specialization within a compact adaptation space.

- We formulate two requirements for a deployment-time proxy—target observability and policy relevance—and instantiate future visual representation prediction as the World-Predictive Interface. Replacing the state-grounding branch of QwenPi with WPI yields QwenWPI, which uses delayed observations as a temporally predictive, label-free objective for prompt-only TTT.
- We expose two challenges introduced by predictive TTT—delayed targets and closed-loop coupling—and develop AGV-TTT to address them. AGV-TTT converts policy-internal attention transitions into isolated shadow proposals, then validates each candidate on matched future observations before atomic commit or rollback.

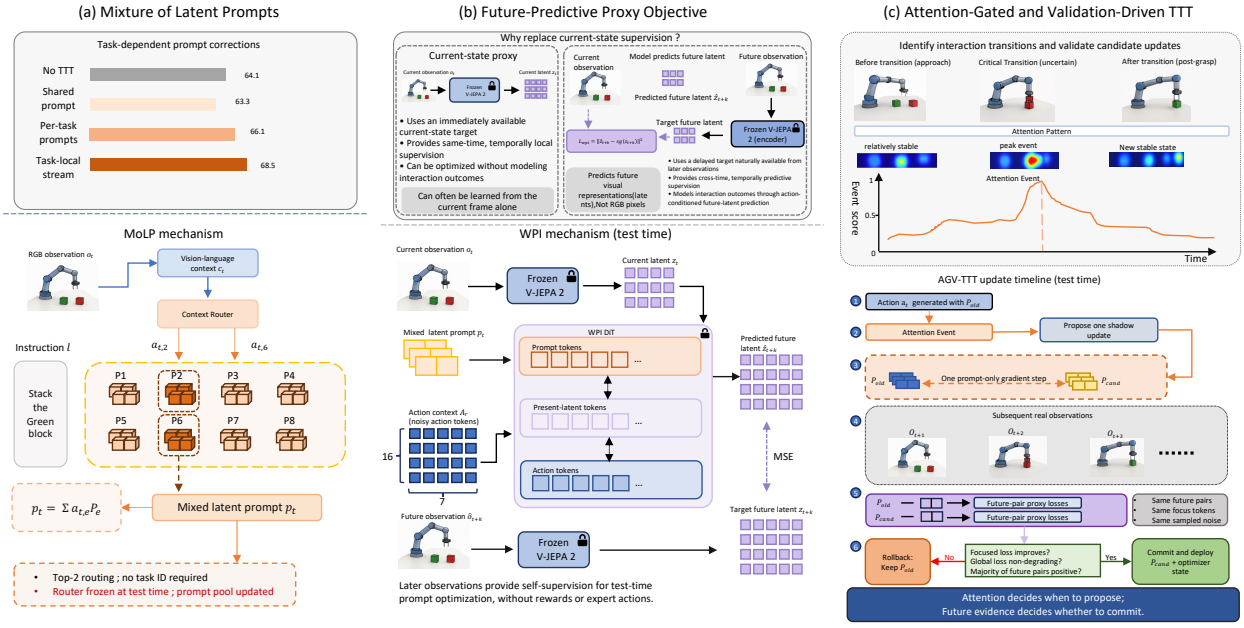


Figure 1: Overview of VANE. (a) MoLP composes a context-dependent latent prompt from a shared prompt bank without requiring a task identifier. (b) The upper comparison contrasts two deployment-observable proxies: state grounding supplies an immediately available but same-time target, whereas WPI uses delayed observations to provide cross-time, action-conditioned supervision of interaction outcomes. The lower diagram instantiates WPI as future visual representation prediction for prompt-only TTT. (c) AGV-TTT uses cross-layer attention redistribution to propose a shadow update and commits it only after paired future validation.

2 RELATED WORK

2.1 VLA Policies and Predictive Interfaces

Generalist vision–language–action policies learn transferable manipulation skills from large-scale robot data and vision–language pretraining [1, 2, 7–9]. Recent policies such as π_0 and StarVLA-α formulate continuous action-chunk generation through flow matching [3, 4]. Our experiments use QwenPi, a Qwen-based flow-matching VLA instantiated following StarVLA-α, as the controlled action-policy backbone. QwenWPI preserves the same VLM and action-policy modules and changes only the proxy interface, isolating the effect of future visual latent prediction.

Predictive representation learning provides supervision by modeling how visual observations evolve. V-JEPA 2 predicts future video representations in latent space, while predictive VLAs incorporate future dynamics into policy pretraining or joint action learning [10–12]. QwenWPI instead keeps the visual target encoder and policy backbone frozen and uses future visual representations as a deployment-time objective for prompt-only optimization.

2.2 Test-Time Training for LLMs, VLMs, and VLA Policies

Test-time learning began in vision with self-supervised training or entropy-based adaptation on unlabeled test samples [5, 13–15]. The idea later reached foundation models: LLMs can train on retrieved neighboring text before inference [16], while VLMs can tune prompts per image using consistent, confident predictions across augmented views [17].

These studies establish label-free adaptation from test inputs, but focus on static prediction, where an update does not determine the next observation.

VLAs add a closed-loop challenge: an update changes subsequent actions and thus the data available for further adaptation. TTT-VLA jointly trains action prediction and state grounding, then adapts only latent prompts at deployment [6]. Concurrently, T^3VF uses predicted–attained future-image pairs and action-variance filtering to adapt visual-foresight policies [18], while reward-based methods require an explicit test-time reward signal [19]. VANE retains latent-prompt adaptation but adds context routing, future-latent supervision, and future validation of shadow updates before closed-loop deployment.

3 METHOD

3.1 Overview

Given an RGB observation o_t and language instruction ℓ, the QwenPi policy predicts an action chunk $\widehat{A}_t \in \mathbb{R}^{16\times 7}$ while being conditioned on an effective latent prompt p_t:

$$\widehat{A}_t = \pi_\Theta(o_t, \ell; p_t). \tag{1}$$

Figure 1 presents VANE as a sequential adaptation pipeline rather than three independent modules: MoLP determines *what* to adapt, WPI supplies the adaptation signal, and AGV-TTT decides *when* to propose an update and *whether* to deploy it.

First, MoLP sparsely routes the current vision–language context over a shared prompt bank to form p_t, without requiring a task identifier. This mitigates cross-task interference and confines adaptation to a structured prompt space. WPI then conditions on p_t, the current visual latent, and action tokens to predict a future visual latent, using later observations as label-free targets. Unlike a current-state proxy, it models action-induced future change without rewards, expert actions, or success labels.

AGV-TTT governs the resulting online update. The deployed prompt P_{old} acts until an attention transition triggers one prompt-only step on a shadow copy, producing P_{cand} while P_{old} remains live. Subsequent observations evaluate both prompts with matched future pairs and controlled inputs. The candidate is committed only if its focused loss improves, its global loss does not degrade, and most future pairs favor it; otherwise, the update is rolled back. Thus, attention decides when to propose, while future evidence decides whether to commit.

During robot-data training, the policy, latent-prompt bank, and router are learned jointly with

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{act}} + \lambda_{\text{wpi}}\mathcal{L}_{\text{wpi}} + \lambda_{\text{lb}}\mathcal{L}_{\text{lb}}, \tag{2}$$

where $\mathcal{L}_{\text{act}}$ is the original flow-matching action loss, $\mathcal{L}_{\text{wpi}}$ is the future visual representation loss, and $\mathcal{L}_{\text{lb}}$ regularizes routing utilization. At deployment, all modules except the latent-prompt bank are frozen, yielding a constrained, verifiable, and reversible adaptation process. For reference, the notation used throughout the method is summarized in Appendix A, and implementation details are provided in Appendix B.

3.2 Context-Routed Mixture of Latent Prompts

A single latent prompt forces heterogeneous task corrections into one shared parameter block, whereas fully independent prompts discard useful cross-task structure. We instead maintain a global latent-prompt bank $\mathcal{P} = \{P_i\}_{i=1}^{E}$ and learn a sparse, context-dependent composition. Let c_t be the last valid token of the final projected VLM representation, which jointly summarizes the current image and instruction. A linear router produces prompt-selection probabilities and the effective prompt:

$$\begin{aligned} q_t &= \mathrm{softmax}(W_r c_t + b_r), \\ \mathcal{S}_t &= \mathrm{TopK}(q_t, K), \\ p_t &= \sum_{i\in\mathcal{S}_t} \frac{q_{t,i}}{\sum_{j\in\mathcal{S}_t} q_{t,j}} P_i. \end{aligned} \tag{3}$$

We use a shared bank of 8 latent prompts with top-2 routing. The bank is not partitioned by task or embodiment, allowing related tasks to reuse prompt components while different vision–language contexts compose distinct prompt mixtures. The routing weights are recomputed for each new control observation and then held fixed throughout the flow-matching iterations used to generate the corresponding action chunk.

The router and latent prompts are trained jointly with the action and proxy objectives, together with a standard load-balancing loss. During test-time training, the router is frozen and only the latent-prompt bank $\mathcal{P}$ is optimized. This

defines a compact adaptation space whose behavior is conditioned on the current vision–language context without requiring an externally provided task identifier. Initialization, load balancing, and optimizer details are provided in Appendix B.

3.3 World-Predictive Interface

A deployment-time proxy must balance target observability with policy relevance. State grounding supplies an immediately available current-state target, but provides same-time, embodiment-specific supervision and can be optimized without modeling the outcome of an interaction. WPI preserves label-free target availability by using later observations and asks the adaptable prompt to predict their frozen visual representations from the current observation and action context. It therefore replaces temporally local state supervision with cross-time, action-conditioned supervision while retaining the original QwenPi action-policy backbone.

A frozen V-JEPA 2 encoder f_ω extracts present and delayed future visual tokens, $Z_t = f_\omega(o_t)$ and $Z_{t+k} = f_\omega(o_{t+k})$. The present tokens, routed prompt, and noised action tokens form the Latent-Action DiT query

$$Q_\tau^{(0)} = [p_t \mid E_z(Z_t) \mid E_a(A_\tau, \tau)]\,. \tag{4}$$

Visual-latent and action tokens exchange information through shared self-attention, so the prediction is conditioned on the policy's action-related context rather than on the current image alone. The slots initialized from Z_t are decoded directly into the delayed representation $\widehat{Z}_{t+k}$ and trained with

$$\mathcal{L}_{\text{wpi}} = \frac{1}{NC}\left\|\widehat{Z}_{t+k} - \text{sg}(Z_{t+k})\right\|_F^2, \tag{5}$$

where N and C are the number and width of the visual tokens. During training, this objective is optimized jointly with action generation. During deployment, the delayed observation supplies the target without rewards, success labels, or expert actions, and gradients are applied only to the latent-prompt bank. The exact flow-matching objective, prediction horizon, and attention-mask implementation are deferred to Appendix B.

3.4 Attention-Gated and Validation-Driven TTT

Online TTT treats every valid proxy pair as equally suitable for an update. Our attention visualizations reveal a clear temporal structure: routine motion occupies a comparatively stable attention regime, whereas contact, grasp closure, and lift-off induce pronounced cross-layer redistribution. AGV-TTT uses this policy-internal transition signal to decide when an update is worth attempting, while reserving future observations to decide whether the attempted update should be deployed.

Endogenous event detection. The live prompt first generates a_t under normal policy inference. During that forward pass, we collect (i) action-to-VLM cross-attention and (ii) action-to-predictive-latent attention. For channel c, denoising step u, and DiT layer l, let $a^c_{t,u,l}$ be the normalized attention distribution. We aggregate its temporal variation as

$$\Delta_t^c = \frac{1}{|\mathcal{U}||\mathcal{L}|}\sum_{u,l}\frac{1}{2}\left\|a^c_{t,u,l} - a^c_{t-1,u,l}\right\|_1. \tag{6}$$

An event is triggered when either channel is anomalous relative to its recent robust baseline. Because detection occurs after a_t is generated, a proposal created by this event cannot retroactively influence the action that triggered it. The robust normalization and thresholds are specified in Appendix C.

Attention-focused shadow proposal. At an event, the predictive-latent attention selects a set $\mathcal{K}_t$ of high-attention visual tokens and assigns normalized weights $F_{t,n}$. With token-wise future-prediction error $e_{t,n}$, the proposal loss is

$$\begin{aligned}\mathcal{L}_{\text{focus},t} &= \sum_{n\in\mathcal{K}_t} F_{t,n}e_{t,n},\\ \mathcal{P}^{\text{cand}} &= \text{Update}(\mathcal{P}^{\text{old}}; \mathcal{L}_{\text{focus},t}).\end{aligned} \tag{7}$$

The selected tokens and weights are detached and frozen for the entire proposal-validation cycle. The update is executed on a shadow copy; the live prompt and optimizer state are immediately restored, so the candidate cannot affect control before validation.

Future validation and atomic deployment. Over the next H_v valid WPI pairs, the old and candidate prompts are evaluated with identical inputs, focus weights, and flow-matching randomness. Let R_{focus} and R_{global} denote their relative improvements in the focused and all-token prediction losses, respectively. A candidate is accepted only when

$$R_{\text{focus}} > 0, \qquad R_{\text{global}} \geq 0,$$
$$\sum_{j=1}^{H_v} \mathbb{I}\Big[R_{\text{focus}}^{(j)} > 0\Big] > \frac{H_v}{2}. \tag{8}$$

Thus, an update must improve the event-focused representation, avoid global regression, and remain beneficial on a temporal majority of subsequent pairs. If accepted, the complete prompt bank and its optimizer state are committed atomically; otherwise, both are discarded. The first $H_v - 1$ validation-frame actions always use the old prompt, while the final validation is resolved before that frame's action is generated. This preserves the causal separation between proposal data, validation evidence, and deployment. Appendix C provides the paired-randomness construction, the state-grounded AGV instance, and the full deployment procedure summarized in Algorithm 1.

4 ANALYSIS AND EXPERIMENTS

We separate two kinds of evidence. The *Diagnostic and Mechanistic Analysis* subsection examines phenomena that motivate or contextualize VANE—task-dependent transfer under shared prompts, policy-internal attention redistribution, and visual-representation shift across evaluation suites. These observations are not treated as direct proof of method efficacy. The *Experimental Evaluation* subsection then evaluates the resulting design through benchmark comparisons, controlled component comparisons, a validation ablation, and optimization-cost measurements.

4.1 Diagnostic and Mechanistic Analysis

4.1.1 Task-Dependent Prompt Interference

We revisit the single-prompt assumption using a fixed checkpoint of the state-grounded policy. Table 1 reports success-rate changes relative to No TTT when a prompt is adapted on different task subsets and then evaluated across all four tasks. The task-specific row uses a separately adapted prompt for each evaluation task; every other row uses one prompt jointly optimized on the named subset. Yellow and blue cells mark the largest gain and degradation in each column, respectively.

Table 1: Task-sharing diagnosis for state-grounded single-prompt Offline TTT on WidowX at a fixed checkpoint.

Adaptation subset	Stack	Carrot	Spoon	Eggplant	Avg.
One-task adaptation					
Task-specific prompts	+2.5	+3.1	0.0	+2.6	+2.0
Two-task shared prompt					
Carrot + Eggplant	+5.2	+10.4	+1.1	0.0	+4.1
Carrot + Spoon	−1.1	+4.2	+2.1	+1.0	+1.5
Spoon + Eggplant	+4.1	+4.2	−2.1	−1.1	+1.3
Stack + Carrot	−2.1	+1.0	+1.1	+1.0	+0.2
Stack + Eggplant	+4.1	+11.5	−6.2	+1.0	+2.6
Stack + Spoon	+4.1	+8.3	0.0	+3.1	+3.9
Three-task shared prompt					
Carrot + Spoon + Eggplant	−2.1	+16.7	−2.1	+3.1	+4.1
Stack + Carrot + Eggplant	+2.0	+5.2	0.0	+3.1	+2.6
Stack + Carrot + Spoon	+3.1	+2.1	+4.2	+2.1	+2.8
Stack + Spoon + Eggplant	+2.0	−1.1	0.0	+1.0	+0.5
All-task shared prompt					
Stack + Carrot + Spoon + Eggplant	−0.6	+2.1	−5.7	+1.0	−0.8

Observation. Prompt adaptation transfers unevenly across tasks, and a single all-task prompt can underperform both the non-adapted and task-specific alternatives. Using a separately adapted prompt for each evaluation task raises the composite four-task average from 64.1% to 66.1%, whereas jointly adapting one shared prompt on all

four tasks lowers it to 63.3%. Intermediate subsets show strongly nonuniform transfer: Carrot + Eggplant and Carrot + Spoon + Eggplant tie for the largest average gain of 4.1 points, while several other subsets improve some evaluation tasks but degrade others. In particular, all-task adaptation reduces Spoon performance by 5.7 points. These outcome-level transfer patterns are consistent with task-dependent interference when heterogeneous contexts share one adapted prompt. They motivate a context-routed latent-prompt bank, while the efficacy of MoLP itself is evaluated separately in Sec. 4.2.3.

4.1.2 Action-to-VLM Attention Changes at Interaction Transitions

We analyze action-to-VLM cross-attention, which captures how action queries attend to VLM image-patch tokens during a rollout. We track changes in this attention distribution across DiT layers and over time. In QwenWPI, AGV additionally monitors action-to-predictive-latent attention during deployment, but that additional channel is not required for the diagnostic below.

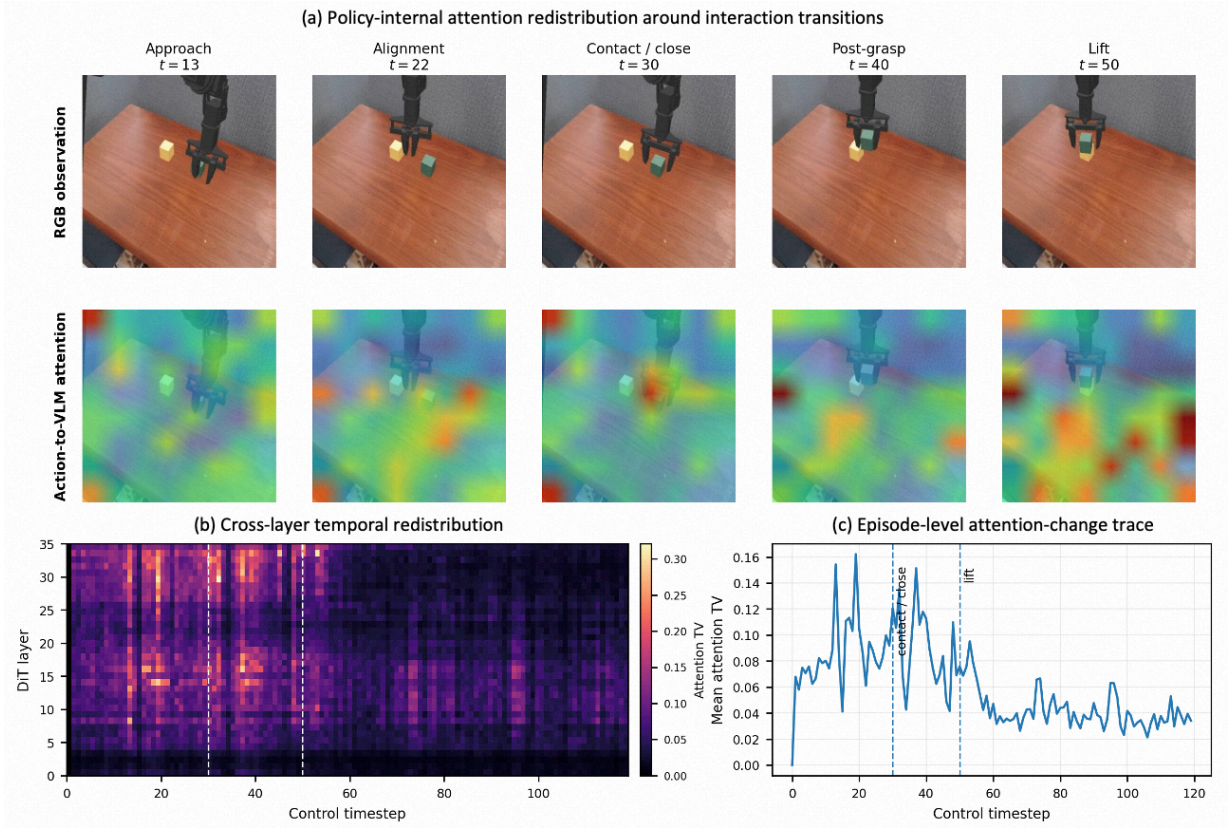


Figure 2: Policy-internal cross-layer attention changes around interaction transitions. (a) RGB observations (upper row) and representative action-query-to-VLM image-patch attention maps (lower row) from a Stack Blocks rollout at a fixed checkpoint; all attention maps share one episode-level scale. (b) Temporal redistribution of the action-to-VLM attention distribution across all 36 DiT layers. (c) The corresponding layer-averaged attention-change trace. Pronounced redistribution occurs around contact/closure and the subsequent post-grasp transition.

Observation. In the illustrated rollout, interaction transitions coincide with pronounced cross-layer attention redistribution. In Figure 2, panel (a) pairs observations from a representative Stack Blocks rollout with the corresponding action-to-VLM attention maps, while panels (b) and (c) show the layer-wise temporal redistribution and its layer average, respectively. Attention varies throughout the rollout, but marked cross-layer redistributions occur around the labeled interaction transitions: gripper–object contact and grasp closure coincide with pronounced changes across multiple DiT layers, followed by another reorganization after grasping. The event score measures redistribution of the full attention distribution rather than the semantic location of its largest spatial response. This single-rollout visualization is illustrative rather than a quantitative event-detection benchmark; it does not establish detector precision or recall. We therefore use attention change only as a label-free, policy-internal cue for *when to consider* a

shadow update. Whether such a proposal is beneficial is a separate question, tested by the future-validation ablation in Sec. 4.2.4.

4.1.3 Visual-Representation Distribution Shift Across Evaluation Suites

Procedure. We use this analysis to compare the magnitudes of visual-representation shift from the robot-data training reference for the WidowX and Google Robot evaluation suites. We first construct a reference set of 1,000 frames sampled equally from RT-1 [7] and Bridge [20]; each evaluation suite contributes another 1,000 frames. Every frame is passed through a fixed checkpoint of the non-adapted QwenPi policy, and the last-layer Qwen3-VL image tokens are mean-pooled into a 2,560-dimensional feature vector.

Next, we fit all preprocessing transforms on the reference set only and compare two feature spaces: the raw standardized features (2,560 dimensions), and a PCA space whose components retain 95% of the reference variance (681 dimensions, denoted PCA-95). For each test suite, we compute the biased RBF-MMD between its feature distribution and the reference distribution. MMD is a kernel two-sample distance: a smaller value means that the test features are closer to the training reference. The RBF bandwidth is selected from the median pairwise distance within the reference set and kept fixed for both test suites.

Finally, we quantify uncertainty by resampling complete episodes 500 times and recomputing each MMD value, rather than resampling individual frames. The brackets in Table 2 are the resulting 95% bootstrap confidence intervals. Repeating the comparison in both feature spaces checks that the ordering is not caused by dimensionality reduction alone.

Table 2: Visual-representation distribution shift from the RT-1/Bridge training mixture.

Test suite	PCA-95 (681-D)	Raw standardized (2560-D)
WidowX	0.330 [0.326, 0.338]	0.348 [0.343, 0.358]
Google Robot	0.262 [0.259, 0.269]	0.262 [0.260, 0.271]

Observation. WidowX is farther than Google Robot from the RT-1/Bridge visual- representation reference in both evaluated feature spaces. Under this common feature extraction and kernel specification, WidowX is farther from the RT-1/Bridge reference than Google Robot in both spaces: 0.330 versus 0.262 after PCA-95 and 0.348 versus 0.262 in the raw standardized space. We use this ordering only as distributional context. MMD does not measure action semantics, embodiment dynamics, or closed-loop difficulty, and it is not used to predict test-time training gains.

4.2 Experimental Evaluation

4.2.1 Experimental Setup

Models and checkpoints. All variants use the same QwenPi backbone and robot-data training recipe introduced in Sec. 3. We jointly train on Bridge [20] and RT-1 [7] with equal sampling probability, and compare state grounding or WPI with either a single latent prompt or the context-routed Mixture of Latent Prompts (MoLP). Unless a diagnostic explicitly uses a fixed checkpoint, results are averaged over the consecutive 50K, 60K, 70K, and 80K checkpoints; evaluation success is never used for checkpoint selection.

Benchmarks and closed-loop evaluation. We evaluate on the WidowX and Google Robot suites of SimplerEnv [21]. WidowX comprises Stack Green Cube on Yellow Cube, Put Carrot on Plate, Put Spoon on Table Cloth, and Put Eggplant in Basket. Google Robot comprises Pick Coke Can, Move Near, Open/Close Drawer, and Put in Drawer under visual matching (VM) and variant aggregation (VA). At each control step, overlapping action chunks are combined using the benchmark's adaptive temporal ensemble, and only the resulting first action is executed before the next observation. Compared variants enumerate the same explicit initial-condition specifications, but we do not enforce a common-random-number rollout protocol across methods. WidowX Overall is the unweighted mean over four tasks; Google Overall is recomputed directly from the reported task-family–setting scores.

Adaptation protocols and notation. *No TTT* leaves the trained prompt unchanged. *Offline TTT* adapts the prompt on a previously collected unlabeled suite buffer and then evaluates the fixed adapted prompt. *Online TTT* immediately deploys each prompt update whenever a valid self-supervised pair becomes available. *AGV-TTT* proposes updates only at attention events and deploys a candidate only after future validation. In the comparison tables, SG-LP denotes the state-grounded single latent prompt used by QwenPi, while FP-LP denotes the future-predictive single latent prompt

used by QwenWPI; FP-LP contains no state expert. Unless stated otherwise, TTT in the public-comparison tables follows the Offline TTT protocol of TTT-VLA [6]. Optimization settings are detailed in Appendix B, and the gating and validation settings in Appendix C.

4.2.2 Comparison with Prior Policies

Table 3: Comparison with prior policies on the SimplerEnv WidowX benchmark.

Method	Carrot	Eggplant	Spoon	Cube	Overall
RT-1-X [8]	4.2	0.0	0.0	0.0	1.1
OpenVLA [2]	0.0	4.1	0.0	0.0	1.0
SpatialVLA [22]	20.8	70.8	20.8	25.0	34.4
Magma [23]	29.2	91.7	37.5	20.8	44.8
Octo-Base [9]	8.3	43.1	12.5	0.0	16.0
Octo-Small [9]	9.7	56.9	47.2	4.2	29.5
RoboVLMs [24]	20.8	79.2	45.8	4.2	37.5
InstructVLA [25]	40.3	94.4	43.1	9.7	46.9
π_0 [3]	36.1	81.9	45.8	26.4	47.6
CogACT [26]	37.5	91.7	58.3	20.8	52.1
ThinkAct [27]	37.5	70.8	58.3	8.7	43.8
TTT-VLA ($\pi_{0.5}$ + SG-LP + TTT) [6]	**74.5**	76.0	71.0	**48.0**	67.4
QwenPi + SG-LP + TTT	58.0	95.6	83.9	18.2	63.9
QwenPi + MoLP + AGV-TTT	64.6	**97.1**	**84.9**	28.4	68.8
QwenWPI + FP-LP + TTT	63.1	90.2	73.9	44.9	68.0
QwenWPI + MoLP + AGV-TTT	68.8	94.5	77.3	44.3	**71.2**

WidowX. Table 3 reports task-level success and the unweighted four-task average. Values for prior policies follow the public comparison in TTT-VLA [6]; our rows average the four consecutive checkpoints. Within the QwenPi family, replacing SG-LP + TTT with MoLP + AGV-TTT raises Overall from 63.9% to 68.8%; within QwenWPI, the corresponding end-to-end comparison improves from 68.0% to 71.2%. The latter also exceeds the published TTT-VLA result of 67.4% by 3.8 points. These comparisons show the performance of the complete VANE configuration against the corresponding TTT baselines. Because each pair changes both prompt structure and update protocol, and the cross-system comparison additionally changes backbone and training data, we defer component attribution to Sec. 4.2.3.

Table 4: Comparison with prior policies on three common Google Robot task families.

Method	Pick Coke Can		Move Near		Open/Close Drawer		Overall
	VM	VA	VM	VA	VM	VA	
TraceVLA [28]	28.0	60.0	53.7	56.4	57.0	31.0	47.7
RT-1-X [8]	56.7	49.0	31.7	32.3	59.7	29.4	43.1
Octo-Base [9]	17.0	0.0	4.2	3.1	22.7	1.1	8.0
OpenVLA [2]	16.3	54.5	46.2	47.7	35.6	17.7	36.3
RoboVLMs (zero-shot) [24]	72.7	68.3	66.3	56.0	26.8	8.5	49.8
RoboVLMs (fine-tuned) [24]	77.3	75.6	61.7	60.0	43.5	10.6	54.8
Emma-X [29]	2.3	5.3	3.3	7.3	18.3	20.5	9.5
π_0 (fine-tuned) [3]	72.7	75.2	65.3	63.7	38.3	25.6	56.8
π_0-FAST (fine-tuned) [30]	75.3	77.6	67.5	68.2	42.9	31.3	60.5
GR00T (fine-tuned) [31]	69.3	46.7	68.7	62.9	35.8	17.5	50.1
TTT-VLA ($\pi_{0.5}$ + SG-LP + TTT) [6]	85.0	79.3	71.7	55.2	60.6	45.8	66.3
QwenPi + SG-LP + TTT	87.6	78.7	79.3	66.9	**68.2**	49.3	71.7
QwenPi + MoLP + AGV-TTT	**94.8**	**85.8**	**87.5**	**71.9**	65.3	55.1	**76.7**
QwenWPI + FP-LP + TTT	84.3	82.4	81.6	68.8	64.8	**59.8**	73.6
QwenWPI + MoLP + AGV-TTT	89.1	83.7	83.3	67.5	55.2	55.4	72.4

Google Robot. The public comparison in TTT-VLA reports Pick Coke Can, Move Near, and Open/Close Drawer but not Put in Drawer. Table 4 therefore uses the same three-task scope and averages the six VM/VA entries for Overall. Within the QwenPi family, the complete VANE configuration raises Overall from 71.7% for SG-LP + TTT to 76.7% for MoLP + AGV-TTT. This is an end-to-end comparison, not an estimate of either component's isolated gain. Within QwenWPI, the corresponding result decreases from 73.6% to 72.4%, indicating that future-predictive prompting is not uniformly superior across this restricted task aggregation. The strongest VANE variant, QwenPi + MoLP + AGV-TTT, exceeds the published TTT-VLA result of 66.3% by 10.4 points, but this cross-system difference does not isolate backbone, training-data, or adaptation effects. We therefore treat the Google comparison as a boundary and scope test rather than evidence of uniform superiority across embodiments.

Table 5: Complete four-task breakdown of our variants on Google Robot.

Method	**Pick Coke Can**		**Move Near**		**Open/Close Drawer**		**Put in Drawer**		**Overall**
	VM	**VA**	**VM**	**VA**	**VM**	**VA**	**VM**	**VA**	
QwenPi + SG-LP + TTT	87.6	78.7	79.3	66.9	**68.2**	49.3	**72.0**	63.4	70.7
QwenPi + MoLP + AGV-TTT	**94.8**	**85.8**	**87.5**	**71.9**	65.3	55.1	60.2	52.9	**71.7**
QwenWPI + FP-LP + TTT	84.3	82.4	81.6	68.8	64.8	**59.8**	19.0	56.0	64.6
QwenWPI + MoLP + AGV-TTT	89.1	83.7	83.3	67.5	55.2	55.4	58.8	**70.4**	70.4

Complete four-task breakdown. Table 5 adds Put in Drawer and averages all eight VM/VA task–setting entries. Within QwenPi, MoLP + AGV-TTT raises Overall from 70.7% to 71.7% relative to SG-LP + TTT. Within QwenWPI, the complete VANE configuration rises markedly from 64.6% to 70.4% after Put in Drawer is included. The largest task-family difference between the two QwenWPI rows occurs on Put in Drawer, where VM/VA increase from $19.0\%/56.0\%$ to $58.8\%/70.4\%$. Together with the restricted three-task result, this shows that the end-to-end benefit of MoLP and AGV-TTT depends on task composition. Because WPI is held fixed within that comparison, the result does not attribute the composition effect to the predictive proxy. We therefore treat Google Robot as a scope test rather than evidence that one proxy or adaptation protocol uniformly dominates every task family.

4.2.3 Controlled Evaluation of VANE Components

Table 6: Controlled comparison on WidowX over four checkpoints.

Proxy	Prompt	No TTT	Offline	Online	AGV
State	Single	64.3	63.9	64.7	65.9
State	MoLP	67.5	68.1	67.8	68.8
WPI	Single	67.5	68.0	67.5	69.0
WPI	MoLP	70.7	69.4	70.3	**71.2**

WidowX. Table 6 provides the factorial comparison used for component attribution. The strongest value is 71.2% for WPI–MoLP with AGV-TTT, but the important evidence is the matched comparison along one factor at a time, summarized below.

Table 7: Controlled comparison on the four-task Google Robot suite.

Proxy	Prompt	No TTT	Offline	Online	AGV
State	Single	73.6	70.7	74.0	74.0
State	MoLP	72.3	71.7	71.6	71.7
WPI	Single	64.2	64.6	64.7	65.3
WPI	MoLP	70.1	70.2	70.5	70.4

Google Robot. Table 7 provides the same factorial comparison on the complete Google suite, where the component ranking changes and the benefits are less uniform. In particular, the strongest configuration remains the state-grounded single-prompt policy, while both WPI rows receive positive but smaller AGV gains over No TTT. Thus, the controlled component benefits are more consistent on WidowX than on Google Robot. These outcomes should be interpreted alongside, rather than explained by, the descriptive representation-shift analysis in Sec. 4.1.3.

Prompt structure. On WidowX, MoLP improves every matched single-prompt configuration when the proxy and adaptation protocol are held fixed. For state grounding, the gains range from $+2.9$ to $+4.2$ points across No TTT, Offline, Online, and AGV; for WPI, the corresponding gains range from $+1.4$ to $+3.2$ points. The positive No-TTT comparisons show that the routed prompt structure already changes the trained policy before deployment-time optimization, while the remaining columns show that the advantage persists under the evaluated TTT protocols. Google Robot is less uniform: MoLP consistently improves the WPI variants but not the state-grounded variants. Together with the diagnostic task-sharing analysis in Sec. 4.1.1, these comparisons separate the motivation for routed prompts from their measured effect.

Predictive proxy. On WidowX, WPI exceeds state grounding in all eight matched prompt–protocol comparisons. The No-TTT gap is already $+3.2$ points for both Single and MoLP, showing that a substantial part of the absolute WPI advantage is present before deployment-time optimization. Under Offline, Online, and AGV, the corresponding absolute gaps remain positive at 1.3–4.1 points. These comparisons therefore support WPI as a stronger proxy-conditioned policy interface on WidowX, but they do not by themselves establish that WPI increases the incremental gain from TTT: improvements relative to each model's own No-TTT baseline remain protocol-dependent. On Google Robot, WPI does not exceed state grounding in any of the eight matched comparisons, further bounding the proxy claim to the evaluated task and embodiment distribution.

Deployment protocol. On WidowX, AGV-TTT is the best evaluated protocol for every proxy–prompt row and the only one that also improves all four rows over their corresponding No-TTT baselines. Relative to No TTT, the gains are $+1.6$, $+1.3$, $+1.5$, and $+0.5$ points for State–Single, State–MoLP, WPI–Single, and WPI–MoLP, respectively; relative to dense Online TTT, AGV improves the same four rows by $+1.2$, $+1.0$, $+1.5$, and $+0.9$ points. On Google Robot, the ordering is not uniform: AGV improves both WPI rows over No TTT, slightly improves State–Single, and degrades State–MoLP. This motivates isolating the validation step from event-triggered sparsity in Sec. 4.2.4.

4.2.4 Is Future Validation Necessary?

Table 8: Ablation of event triggering and future validation on single-prompt QwenWPI.

Online protocol	Event gate	Future validation	Success
Online TTT	–	–	67.5
Event + Accept-All	✓	–	67.3
AGV-TTT	✓	✓	**69.0**

Result. To separate future validation from event triggering, *Accept-All* retains the same attention detector and shadow proposal as AGV-TTT but commits every completed candidate without evaluating it on subsequent observations. As shown in Table 8, Event + Accept-All reaches 67.3%, slightly below Online TTT at 67.5%. Future validation raises success to 69.0%, a 1.7-point improvement over Accept-All. Event-triggered sparsity alone therefore does not explain the gain. The 1.7-point gap isolates a measurable benefit of future validation within the same event-gated proposal pipeline, consistent with validation filtering unfavorable candidates before deployment.

4.2.5 Optimization Efficiency

Table 9: Per-checkpoint optimization and validation cost for single-prompt QwenWPI, averaged over four checkpoints.

Protocol	Backward	Validation forward	Success
Online TTT	45,824	–	67.5
AGV-TTT	612	4,831	**69.0**

Result. Online TTT performs a backward update whenever a valid WPI pair becomes available, whereas AGV-TTT proposes a shadow update only at an attention event. For the same single-prompt QwenWPI policy over four checkpoints, Table 9 shows that AGV-TTT performs an average of 612 candidate backward passes per checkpoint evaluation, compared with $45{,}824$ for Online TTT. It triggers backward optimization on only 1.3% of eligible frames and reduces backward calls by 98.7% (approximately $75\times$), while improving mean success from 67.5% to 69.0%.

AGV-TTT additionally performs $4{,}831$ forward-only proxy evaluations for future validation. Only 19.4% of proposals are accepted, so most shadow proposals never enter the deployed policy. This selectivity is functionally important:

Event + Accept-All obtains 67.3%, whereas validated AGV-TTT reaches 69.0%. The gain therefore comes from event-triggered proposals together with subsequent validation, not from reducing update frequency alone. We report optimization counts rather than converting them into an unsupported wall-clock speedup.

5 CONCLUSION

We presented VANE, a test-time training framework for VLAs that combines a context-routed MoLP, future visual representation prediction, and attention-gated, validation-driven updates. Attention redistribution identifies informative interaction transitions for proposing shadow updates, while subsequent observations determine whether each candidate is committed or rolled back.

On SimplerEnv WidowX, the learned WPI–MoLP interface improves the non-adapted policy from 64.3% to 70.7%, and AGV-TTT further raises success to 71.2%. AGV-TTT is the only evaluated protocol that improves checkpoint-averaged performance across all four proxy–prompt configurations. In the single-prompt QwenWPI efficiency study, it reduces backward updates by 98.7% relative to Online TTT. Google Robot results further show that adaptation gains remain task- and embodiment-dependent, alongside measurable differences in visual-representation shift. These findings support a context-routed, temporally predictive adaptation interface on WidowX and an explicit proposal–validation mechanism for online updates, while the Google results bound the universality of these gains.

References

[1] A. Brohan *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.

[2] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[3] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[4] J. Ye, N. Gao, S. Yang, J. Zheng, Z. Wang, Y. Chen, P. Chen, Y. Chen, S. Liu, and J. Jia, "StarVLA-α: Reducing complexity in vision-language-action systems," *arXiv preprint arXiv:2604.11757*, 2026.

[5] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt, "Test-time training with self-supervision for generalization under distribution shifts," in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 9229–9248.

[6] W. Zhang, J. Li, S. Yang, S. Chen, J. Liu, L. Liu, and X. Ma, "TTT-VLA: Test-time latent prompt optimization for vision-language-action models," *arXiv preprint arXiv:2606.03127*, 2026.

[7] A. Brohan *et al.*, "RT-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.

[8] Open X-Embodiment Collaboration, A. O'Neill *et al.*, "Open X-Embodiment: Robotic learning datasets and RT-X models," *arXiv preprint arXiv:2310.08864*, 2023.

[9] Octo Model Team, D. Ghosh *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[10] M. Assran *et al.*, "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.

[11] X. Xu, H. Li, J. Ye, Y. Chen, J. Zeng, X. Chen, L. Xu, D. Lin, W. Li, and J. Pang, "FutureVLA: Joint visuomotor prediction for vision-language-action model," *arXiv preprint arXiv:2603.10712*, 2026.

[12] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, Y. Shen, and Y. Xu, "Causal world modeling for robot control," *arXiv preprint arXiv:2601.21998*, 2026.

[13] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," *arXiv preprint arXiv:2006.10726*, 2020.

[14] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," *arXiv preprint arXiv:2203.13591*, 2022.

[15] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, "Efficient test-time model adaptation without forgetting," *arXiv preprint arXiv:2204.02610*, 2022.

[16] M. Hardt and Y. Sun, "Test-time training on nearest neighbors for large language models," in *International Conference on Learning Representations*, 2024.

[17] M. Shu, W. Nie, D.-A. Huang, Z. Yu, T. Goldstein, A. Anandkumar, and C. Xiao, "Test-time prompt tuning for zero-shot generalization in vision-language models," in *Advances in Neural Information Processing Systems*, 2022.

[18] S. Park, W. Kim, Y. In, S. Kim, H. Kang, and C. Park, "Test-time training for visual foresight vision-language-action models," *arXiv preprint arXiv:2605.08215*, 2026.

[19] C. Liu, Y. Liu, T. Wang, Q. Zhuang, J. C. Liang, W. Yang, R. Xu, Q. Wang, D. Liu, and C. Han, "On-the-fly VLA adaptation via test-time reinforcement learning," *arXiv preprint arXiv:2601.06748*, 2026.

[20] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine, "BridgeData V2: A dataset for robot learning at scale," *arXiv preprint arXiv:2308.12952*, 2023.

[21] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao, "Evaluating real-world robot manipulation policies in simulation," *arXiv preprint arXiv:2405.05941*, 2024.

[22] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li, "SpatialVLA: Exploring spatial representations for vision-language-action model," *arXiv preprint arXiv:2501.15830*, 2025.

[23] J. Yang, R. Tan, Q. Wu, R. Zheng, B. Peng, Y. Liang, Y. Gu, M. Cai, S. Ye, J. Jang, Y. Deng, and J. Gao, "Magma: A foundation model for multimodal AI agents," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 14 203–14 214.

[24] X. Li, P. Li, M. Liu, D. Wang, J. Liu, B. Kang, X. Ma, T. Kong, H. Zhang, and H. Liu, "Towards generalist robot policies: What matters in building vision-language-action models," *arXiv preprint arXiv:2412.14058*, 2024.

[25] S. Yang, H. Li, B. Wang, Y. Chen, Y. Tian, T. Wang, H. Wang, F. Zhao, Y. Liao, and J. Pang, "InstructVLA: Vision-language-action instruction tuning from understanding to manipulation," *arXiv preprint arXiv:2507.17520*, 2025.

[26] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, X. Wang, B. Liu, J. Fu, J. Bao, D. Chen, Y. Shi, J. Yang, and B. Guo, "CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," *arXiv preprint arXiv:2411.19650*, 2024.

[27] C.-P. Huang, Y.-H. Wu, M.-H. Chen, Y.-C. F. Wang, and F.-E. Yang, "ThinkAct: Vision-language-action reasoning via reinforced visual latent planning," in *Advances in Neural Information Processing Systems*, 2025.

[28] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang, "TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies," in *International Conference on Learning Representations*, 2025.

[29] Q. Sun, P. Hong, T. D. Pala, V. Toh, U.-X. Tan, D. Ghosal, and S. Poria, "Emma-X: An embodied multimodal action model with grounded chain of thought and look-ahead spatial reasoning," *arXiv preprint arXiv:2412.11974*, 2024.

[30] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, "FAST: Efficient action tokenization for vision-language-action models," in *Proceedings of Robotics: Science and Systems*, 2025.

[31] NVIDIA, J. Bjorck, F. Castaneda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox *et al.*, "GR00T N1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

APPENDIX

A. Notation

The table summarizes symbols reused across the method and deployment procedure; quantities local to a single derivation are defined where they are introduced.

Symbol	Definition
o_t, ℓ	RGB observation and language instruction at control step t.
$\widehat{A}_t, a_t$	Policy-predicted action chunk and executed action at control step t.
Θ	Policy parameters, frozen during deployment-time adaptation.
$\mathcal{P}, P_i, p_t$	Latent-prompt bank, prompt component i, and effective routed prompt.
$c_t, q_t, \mathcal{S}_t$	Router context, prompt-selection probabilities, and selected set.
E, K	Number of banked latent prompts and selected prompts.
f_ω, Z_t, Z_{t+k}	Frozen visual encoder and its present and delayed future visual tokens.
$\mathcal{L}_{\mathrm{act}}, \mathcal{L}_{\mathrm{wpi}}, \mathcal{L}_{\mathrm{lb}}$	Action, future-prediction, and router load-balancing losses.
$a^c_{t,u,l}, \Delta^c_t$	Channel-c attention at step t, flow step u, and layer l, and its aggregated temporal change.
$\mathcal{K}_t, F_{t,n}, e_{t,n}$	Focused token set, normalized weight, and token-wise prediction error.
$\mathcal{L}_{\mathrm{focus}}$	Attention-weighted prediction loss over focused tokens.
$\mathcal{P}^{\mathrm{old}}, \mathcal{P}^{\mathrm{cand}}$	Live and shadow candidate prompt banks.
H_v, R_x	Validation horizon and relative improvement for loss x.

B. Training and Architectural Details

Flow-matching action objective. For the clean action chunk $A_1 \in \mathbb{R}^{16\times 7}$, we sample $A_0 \sim \mathcal{N}(0, I)$ and $u \sim \mathrm{Beta}(1.5, 1.0)$, then set

$$\tau = \frac{\eta - u}{\eta}, \quad \eta = 0.999, \qquad A_\tau = (1-\tau)A_0 + \tau A_1. \tag{9}$$

Let H_a and d_a denote the action-chunk horizon and per-step action dimension. The action-branch output $\widehat{v}_\Theta$ predicts the constant target velocity $A_1 - A_0$:

$$\mathcal{L}_{\mathrm{act}} = \mathbb{E}\left[\frac{1}{H_a d_a}\left\|\widehat{v}_\Theta(A_\tau, \tau) - (A_1 - A_0)\right\|_F^2\right]. \tag{10}$$

All action steps and dimensions are weighted equally. Repeated noise-time samples provide a Monte Carlo estimate of the same objective.

Router regularization and initialization. For a batch of samples, let f_i be the fraction routed to latent prompt i and $\bar{q}_i$ its mean router probability. We use

$$\mathcal{L}_{\mathrm{lb}} = E\sum_{i=1}^{E} f_i \bar{q}_i, \quad \mathcal{L}_{\mathrm{train}} = \mathcal{L}_{\mathrm{act}} + 0.1\mathcal{L}_{\mathrm{wpi}} + 0.01\mathcal{L}_{\mathrm{lb}}. \tag{11}$$

The latent-prompt bank has shape $[8, 16, 1024]$; its eight prompt components are independently initialized from $\mathcal{N}(0, 0.02^2)$. The linear router uses the PyTorch default initialization, and the one-stage model is not restored from an earlier prompt or router checkpoint.

Latent-Action DiT visibility. The WPI query is ordered as [prompt | present visual latent | action], with bidirectional interaction between present-latent and action tokens. Following the latent-prompt visibility strategy of TTT-VLA [6], the direct action-query-to-prompt-key connection is masked during prompt learning and restored during policy inference and prompt-only TTT. This inherited design prevents the action branch from relying prematurely on the latent prompt during training, while allowing the adapted prompt to modulate the frozen action branch at deployment. QwenWPI changes the proxy interface by introducing predictive visual-latent tokens, but does not alter this prompt-to-action visibility strategy. Prompt/visual and action tokens use separate feed-forward branches while sharing attention. V-JEPA 2 remains frozen, the future offset is $k = 8$, and the WPI loss averages ordinary squared error over all visual tokens and channels.

Prompt-only optimization. At test time, the full $[8, 16, 1024]$ prompt bank is registered as one AdamW parameter while the router and policy remain frozen. Current proxy gradients are sparse in the selected prompt slices, but decay and historical moments are maintained for the complete prompt bank. AGV therefore snapshots, commits, and rolls back both the complete prompt bank and optimizer state.

Test-time optimization settings. Offline TTT performs suite-wide prompt-only AdamW optimization with learning rate 10^{-5} and batch size 64 for 500 steps on WidowX and 1000 steps on Google Robot, after which the adapted prompt is fixed during evaluation. Dense Online TTT uses batch size 1 and takes one AdamW step with learning rate 10^{-7} whenever a valid WPI pair arrives. Each AGV proposal consists of one shadow AdamW step with learning rate 10^{-7}, $\beta = (0.9, 0.999)$, and weight decay 0.01; the candidate prompt and optimizer state are committed only after future validation. These settings update latent prompts only; the router and policy backbone remain frozen.

C. AGV-TTT Implementation Details

Robust event gate. The change score in Eq. (6) is normalized against a rolling history $\mathcal{H}_t^c$:

$$r_t^c = \frac{\Delta_t^c - \mathrm{median}(\mathcal{H}_t^c)}{\max(1.4826\,\mathrm{MAD}(\mathcal{H}_t^c), \epsilon)}. \tag{12}$$

Here MAD denotes median absolute deviation and $\epsilon > 0$ is a numerical-stability constant. An event requires both an absolute and a robust threshold in either channel. Warm-up, cooldown, and a single-pending-candidate guard suppress startup noise and overlapping proposals.

Focused token construction. Predictive-latent attention is normalized per layer and flow step and then aggregated over selected late DiT layers. Let $\mathcal{S}$ denote the selected flow-step–layer pairs and $a_{t,u,l,n}^{\mathrm{jepa}}$ the predictive-latent attention weight for visual token n among N visual tokens:

$$\begin{aligned} A_n &= \frac{1}{|\mathcal{S}|} \sum_{(u,l)\in\mathcal{S}} \frac{a_{t,u,l,n}^{\mathrm{jepa}}}{\sum_{j=1}^{N} a_{t,u,l,j}^{\mathrm{jepa}}}, \\ F_n &= \frac{A_n}{\sum_{j\in\mathcal{K}_t} A_j}, \qquad n \in \mathcal{K}_t. \end{aligned} \tag{13}$$

Here A_n is the aggregated event-frame attention and F_n is its normalized weight over $\mathcal{K}_t$, corresponding to $F_{t,n}$ in the main text. The main experiments use the top 32 tokens. Single-prompt WPI and WPI + MoLP aggregate the last four and eight layers, respectively. The indices and weights are created from the live event-frame attention, detached, and reused for proposal training and all validation pairs.

Paired future validation. For each validation pair, old and candidate prompts receive identical present and future latents, focus weights, Gaussian action noise, and flow time. CPU and CUDA random states are reset before each branch and restored afterward. Since no expert action is available, both branches use the same zero action anchor; only future-latent prediction errors are compared. For $x \in \{\mathrm{focus}, \mathrm{global}\}$,

$$R_x = \frac{\overline{\mathcal{L}}_x^{\mathrm{old}} - \overline{\mathcal{L}}_x^{\mathrm{cand}}}{\max(|\overline{\mathcal{L}}_x^{\mathrm{old}}|, \epsilon)}. \tag{14}$$

The overline denotes the mean over the collected validation pairs, and the same numerical-stability constant ϵ is used in the denominator. The WPI setting uses $H_v = 4$, focus weight $\lambda_f = 1$, zero improvement margin, and zero global-degradation tolerance. Finite-loss, parameter, and step checks are applied before commit. A candidate that cannot finish validation within an episode is discarded.

Algorithm 1: WPI-AGV-TTT at deployment time

Initialize live prompt/optimizer $(\mathcal{P}, m)$ and no candidate
for each control step t **do**
  if a candidate is pending and a new WPI pair is available **then**
    Evaluate old and candidate with paired randomness
    if H_v validation pairs have been collected **then**
      Commit iff Eq. (8) holds; otherwise retain live state
      Clear the candidate
    end if
  end if
  Generate a_t with the live prompt and capture attention
  if an event is detected and no candidate is pending **then**
    Freeze the attention-derived focus specification
    Take one shadow AdamW step on $\mathcal{L}_{\text{focus}}$
    Restore live state and mark the candidate as pending
  end if
  Execute the already generated action a_t
end for

State-grounded AGV. For the state-grounded baseline, AGV uses only action-to-VLM attention, the state-grounding proxy, and a three-pair majority validation. Its acceptance margin is 10^{-6}, whereas WPI uses zero. This variant tests whether the proposal-validation-commit abstraction transfers to another proxy; the full VANE method is the dual-attention WPI instance.